

AMAZON E-COMMERCE MANAGEMENT SYSTEM

PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

REVIEW AND RATING MANAGEMENT

1. Introduction

The Review and Rating module is an important part of the Amazon E-Commerce Management System. It allows customers to provide feedback about the products they purchase. Customers can give a rating from 1 to 5 and write a review describing their experience with the product.

This module helps the system store customer feedback, retrieve product reviews, calculate average product ratings, and identify highly rated products.

1.REVIEW TABLE

Column Name	Data Type	Constraint	Description
REVIEW_ID	NUMBER	Primary Key	Unique review ID
PRODUCT_ID	NUMBER	Foreign Key	Product being reviewed
CUSTOMER_ID	NUMBER	Foreign Key	Customer who gives the review
ORDER_ID	NUMBER	Foreign Key	Related order
REVIEW_TEXT	VARCHAR2(500)	—	Customer feedback
REVIEW_DATE	DATE	Default SYSDATE	Date of review

2.RATING TABLE

Column Name	Data Type	Constraint	Description
RATING_ID	NUMBER	Primary Key	Unique rating ID
PRODUCT_ID	NUMBER	Foreign Key	Rated product
CUSTOMER_ID	NUMBER	Foreign Key	Customer who gives the rating
RATING	NUMBER(1)	CHECK (1–5)	Rating given by customer
RATING_DATE	DATE	Default SYSDATE	Date of rating

3.CREATE REVIEW TABLE:

```
SQL> CREATE TABLE REVIEW (  
 2     REVIEW_ID NUMBER PRIMARY KEY,  
 3     PRODUCT_ID NUMBER NOT NULL,  
 4     CUSTOMER_ID NUMBER NOT NULL,  
 5     ORDER_ID NUMBER NOT NULL,  
 6     REVIEW_TEXT VARCHAR2(500),  
 7     REVIEW_DATE DATE DEFAULT SYSDATE  
 8 );
```

Table created.

4.CREATE RATING TABLE

```
SQL> CREATE TABLE RATING (  
 2     RATING_ID NUMBER PRIMARY KEY,  
 3     PRODUCT_ID NUMBER NOT NULL,  
 4     CUSTOMER_ID NUMBER NOT NULL,  
 5     RATING NUMBER(1) CHECK (RATING BETWEEN 1 AND 5),  
 6     RATING_DATE DATE DEFAULT SYSDATE  
 7 );
```

Table created.

5.INSERTING VALUES

```
SQL> INSERT INTO REVIEW  
 2 (REVIEW_ID, PRODUCT_ID, CUSTOMER_ID, ORDER_ID, REVIEW_TEXT)  
 3 VALUES  
 4 (1, 1, 1, 1, 'Excellent product and very useful');
```

1 row created.

```
SQL>  
SQL> INSERT INTO REVIEW  
 2 (REVIEW_ID, PRODUCT_ID, CUSTOMER_ID, ORDER_ID, REVIEW_TEXT)  
 3 VALUES  
 4 (2, 1, 2, 2, 'Good product and worth the price');
```

1 row created.

```
SQL>  
SQL> INSERT INTO REVIEW  
 2 (REVIEW_ID, PRODUCT_ID, CUSTOMER_ID, ORDER_ID, REVIEW_TEXT)  
 3 VALUES  
 4 (3, 2, 3, 3, 'Amazing quality and fast delivery');
```

1 row created.

```
SQL>  
SQL> INSERT INTO REVIEW  
 2 (REVIEW_ID, PRODUCT_ID, CUSTOMER_ID, ORDER_ID, REVIEW_TEXT)  
 3 VALUES  
 4 (4, 2, 1, 4, 'Good quality product');
```

1 row created.

```
SQL>  
SQL> INSERT INTO REVIEW  
 2 (REVIEW_ID, PRODUCT_ID, CUSTOMER_ID, ORDER_ID, REVIEW_TEXT)  
 3 VALUES  
 4 (5, 3, 2, 5, 'Product is okay but could be better');
```

1 row created.

6.INSERT RATINGS:

```
SQL> INSERT INTO RATING
  2 (RATING_ID, PRODUCT_ID, CUSTOMER_ID, RATING)
  3 VALUES
  4 (1, 1, 1, 5);

1 row created.

SQL>
SQL> INSERT INTO RATING
  2 (RATING_ID, PRODUCT_ID, CUSTOMER_ID, RATING)
  3 VALUES
  4 (2, 1, 2, 4);

1 row created.

SQL>
SQL> INSERT INTO RATING
  2 (RATING_ID, PRODUCT_ID, CUSTOMER_ID, RATING)
  3 VALUES
  4 (3, 2, 3, 5);

1 row created.

SQL>
SQL> INSERT INTO RATING
  2 (RATING_ID, PRODUCT_ID, CUSTOMER_ID, RATING)
  3 VALUES
  4 (4, 2, 1, 4);

1 row created.

SQL>
SQL> INSERT INTO RATING
  2 (RATING_ID, PRODUCT_ID, CUSTOMER_ID, RATING)
  3 VALUES
  4 (5, 3, 2, 3);
```

7.RETRIEVE PRODUCT REVIEW DETAILS:

```
SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     C.ID AS CUSTOMER_ID,
  5     C.NAME AS CUSTOMER_NAME,
  6     R.REVIEW_TEXT,
  7     R.REVIEW_DATE
  8 FROM REVIEW R
  9 JOIN PRODUCT P
 10     ON R.PRODUCT_ID = P.PRODUCT_ID
 11 JOIN CUSTOMER C
 12     ON R.CUSTOMER_ID = C.ID
 13 ORDER BY R.REVIEW_DATE DESC;
```

8. CALCULATE AVERAGE PRODUCT RATINGS:

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating
2  FROM RATING
3  GROUP BY Product_ID;
```

PRODUCT_ID	AVERAGE_RATING
1	4.5
2	4.5
3	3

9.IDENTIFY HIGHLY RATED PRODUCTS:

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating
2  FROM RATING
3  GROUP BY Product_ID
4  HAVING AVG(Rating) >= 4;
```

PRODUCT_ID	AVERAGE_RATING
1	4.5
2	4.5